

IMPLEMENTASI ALGORITMA GREEDY PADA BOT ROBOCODE TANK ROYALE



Disusun untuk memenuhi tugas mata kuliah
Strategi Algoritma

Disusun Oleh:

Jhon Kevin Hamonangan Tambun	124140073
Aditya Kristian Novalino	124140079
Muhammad Ghifar	124140091

Program Studi Teknik Infomatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera
2026

BAB 1	
DESKRIPSI TUGAS.....	4
1.1 Latar Belakang.....	4
1.2 Deskripsi Tugas.....	4
Starter Pack.....	9
1.3 Tujuan.....	9
BAB 2	
LANDASAN TEORI.....	10
2.1 Algoritma Greedy.....	10
2.2 Robocode Tank Royale.....	11
2.3 Cara Kerja Program.....	11
2.4 Implementasi Algoritma Greedy Pada Bot.....	12
2.5 Heuristic pada Algoritma Greedy.....	12
2.6 Bahasa Pemrograman yang Digunakan.....	13
2.7 Cara Menjalankan Bot.....	13
BAB 3	
APLIKASI STRATEGI GREEDY.....	14
3.1 Mapping Persoalan Robocode ke Elemen Algoritma Greedy.....	14
3.1.1 Himpunan Kandidat.....	14
3.1.2 Himpunan Solusi.....	14
3.1.3 Fungsi Seleksi.....	14
3.1.4 Fungsi Kelayakan.....	14
3.1.5 Fungsi Objektif.....	15
3.1.6 Fungsi Solusi.....	15
3.2 Alternatif Strategi Greedy.....	15
3.2.1 Aggressive Greedy (Bot ClosestHunter).....	15
3.2.2 Survival Greedy (Bot Kabur).....	15
3.2.3 Defensive Evasive Strategy (Bot Push).....	16
3.2.4 Predictive Evasive Strategy (Bot NAHUBB).....	16
3.3 Analisis Efisiensi dan Efektivitas.....	17
3.4 Strategi yang Dipilih.....	17
BAB 4	
IMPLEMENTASI DAN PENGUJIAN.....	18
4.1 Implementasi.....	18
4.1.1 Implementasi Metode DoMovement pada Kelas Bot NAHUBB.....	18
4.1.2 Implementasi Metode DoGunAndRadar pada kelas Bot NAHUBB.....	19
4.1.3 Implementasi fungsi CalculateFirePower pada kelas Bot NAHUBB.....	20
4.1.4 Implementasi fungsi IsNearWall pada kelas Bot NAHUBB.....	20
4.2 Struktur Data.....	20
4.3 Analisis Pengujian.....	22
4.3.1 Pengujian Ke-1.....	23
4.3.2 Pengujian Ke-2.....	24
4.3.3 Pengujian Ke-3.....	26
4.4 Analisis Hasil Pengujian.....	27
BAB 5	

KESIMPULAN DAN SARAN.....	28
5.1 Kesimpulan.....	28
5.2 Saran.....	28
LAMPIRAN.....	29
DAFTAR PUSTAKA.....	30

BAB 1

DESKRIPSI TUGAS

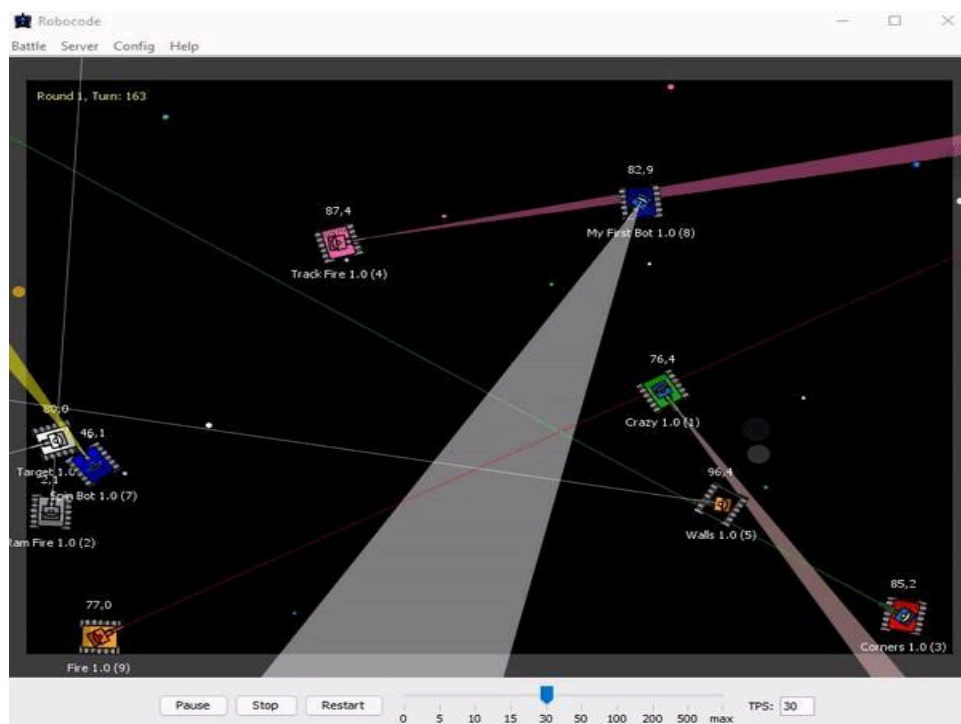
1.1 Latar Belakang

Perkembangan kecerdasan buatan dalam bidang permainan digital semakin berkembang pesat. Salah satu penerapan kecerdasan buatan dapat ditemukan pada game simulasi strategi seperti Robocode Tank Royale. Pada permainan ini, bot dikendalikan oleh program yang dapat menentukan tindakan secara otomatis, seperti bergerak, menghindari serangan, dan menembak musuh.

Dalam pengambilan keputusan secara cepat, algoritma greedy menjadi salah satu pendekatan yang dapat digunakan. Algoritma greedy bekerja dengan memilih keputusan terbaik pada setiap langkah berdasarkan kondisi saat itu tanpa mempertimbangkan seluruh kemungkinan yang ada.

Melalui tugas ini, algoritma greedy diterapkan pada bot Robocode Tank Royale untuk mengetahui bagaimana strategi greedy dapat digunakan dalam pertandingan antar bot.

1.2 Deskripsi Tugas



Gambar 1. Robocode Tank Royale

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

1. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewati turn tersebut. Jika bot melewati turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

2. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

3. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

4. Panas Meriam (Gun Heat)

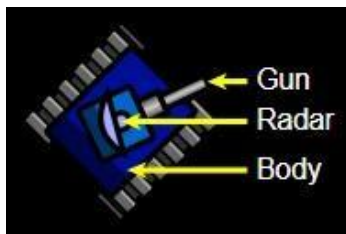
Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

5. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain. Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

6. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank. *Gun* digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*. *Radar* digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

7. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak tergantung pada kecepatan bot saat itu.

8. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

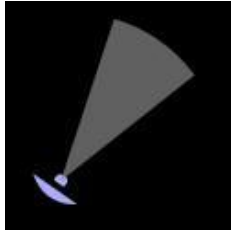
Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar *damage*** yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.

- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

Untuk informasi lebih lengkap, silahkan buka dokumentasi Tank Royale pada link [berikut](#).

Starter Pack

Untuk tugas besar ini, game engine yang akan digunakan sudah dimodifikasi oleh asisten.

Berikut adalah beberapa perubahan yang dibuat oleh asisten dari game engine Tank Royale:

- **Theme GUI:** diubah menjadi light theme
- **Skor & Energi:** masing-masing bot ditampilkan di samping arena permainan agar lebih mudah diamati saat pertarungan

Turn Limit: Durasi pertarungan tidak akan bergantung pada banyak ronde. Pada game engine ini, pertarungan akan berakhir apabila banyaknya turn sudah mencapai batas tertentu. Apabila batasan ini tercapai, ronde otomatis langsung berakhir dan pemenang pertarungan akan ditampilkan. Turn Limit dapat diatur pada menu “setup rules”. Source Code untuk game engine dan template bot telah disediakan pada tautan berikut.

[tubes1-if2211-starter-pack](#)

Adapun panduan mengenai cara menjalankan game engine, membuat bot, dan melihat referensi API dapat dilihat melalui tautan berikut: [Starter Guide](#).

1.3 Tujuan

Tujuan dari tugas ini adalah:

1. Memahami konsep algoritma greedy.
2. Mengimplementasikan algoritma greedy pada bot Robocode Tank Royale.
3. Menganalisis efektivitas strategi greedy dalam pertandingan.
4. Membandingkan beberapa heuristic greedy untuk menentukan strategi terbaik.

BAB 2

LANDASAN TEORI

2.1 Algoritma Greedy

Algoritma Greedy merupakan metode yang digunakan untuk menyelesaikan masalah optimasi dengan cara memilih solusi terbaik pada setiap langkah secara bertahap (*step by step*). Pendekatan ini dilakukan dengan mengambil keputusan yang dianggap paling menguntungkan pada saat itu, dengan tujuan memperoleh hasil yang optimal atau mendekati optimal sesuai kriteria yang telah ditentukan. Dalam penerapannya, algoritma ini berfokus pada pemilihan kandidat solusi yang memenuhi syarat tertentu sehingga proses pencarian solusi dapat dilakukan dengan lebih sederhana dan efisien.

Salah satu penerapan Algoritma Greedy adalah dalam pencarian lintasan terpendek menuju suatu tujuan. Dengan metode ini, sistem dapat menentukan jalur yang dianggap paling baik pada setiap langkah untuk mencapai tujuan yang diinginkan. Prinsip utama dari algoritma ini adalah memilih opsi terbaik yang tersedia saat itu tanpa mempertimbangkan seluruh kemungkinan di masa depan. Oleh karena itu, meskipun metode greedy sering menghasilkan solusi yang cepat, terdapat kemungkinan solusi yang diperoleh bukan solusi paling optimal secara global.

Dalam proses penyelesaiannya, Algoritma Greedy bekerja melalui beberapa tahapan. Tahap pertama adalah menentukan pilihan terbaik pada kondisi saat ini berdasarkan kriteria tertentu. Selanjutnya, pilihan tersebut dijadikan bagian dari solusi sementara dan proses diulang pada langkah berikutnya hingga solusi lengkap diperoleh. Pendekatan ini mengandalkan asumsi bahwa keputusan lokal terbaik pada setiap langkah akan menghasilkan solusi global yang baik. Dengan demikian, Algoritma Greedy banyak digunakan karena memiliki proses yang sederhana, efisien, dan mampu memberikan solusi yang cukup optimal pada berbagai permasalahan optimasi.

Adapun keunggulan algoritma greedy adalah :

- Implementasi Sederhana
- Eksekusi Cepat
- Cocok untuk pengambilan keputusan real-time

Namun, algoritma greedy juga punya kelemahan, yaitu tidak selalu menghasilkan solusi global yang paling optimal pada semua kasus, karena hanya mempertimbangkan kondisi saat ini tanpa memprediksi kemungkinan di masa depan.

Dalam implementasi pada permainan Robocode Tank Royale, algoritma greedy digunakan saat menentukan tindakan bot secara cepat berdasar kondisi arena pertandingan yang sedang berlangsung.

2.2 Robocode Tank Royale

Robocode Tank Royale merupakan permainan simulasi pertarungan tank berbasis pemrograman yang dikembangkan sebagai media pembelajaran algoritma, kecerdasan buatan, dan strategi pengambilan keputusan.

Pada permainan ini, pengguna tidak mengendalikan tank secara langsung, namun membuat program bot yang dapat bertarung secara otomatis di dalam arena. Setiap bot diprogram menggunakan bahasa pemrograman tertentu seperti C#.

Bot dalam robocode tank royale mempunyai beberapa keahlian utama, yaitu :

- Bergerak maju, mundur, dan berputar
- Memindai posisi musuh menggunakan radar
- Mengatur strategi pergerakan dan penembakan
- Menghindari serangan lawan
- Mengarahkan meriam ke target
- Menembakkan peluru

Pertandingan berlangsung secara otomatis sampai hanya tersisa 1 bot yang dapat bertahan atau waktu pertandingan selesai. Pemenang dapat ditentukan berdasar skor yang diperoleh :

- Jumlah musuh yang berhasil dikalahkan
- Akurasi serangan
- Ketahanan hidup bot
- Efektivitas strategi individu

Robocode sering digunakan untuk media pembelajaran karena bisa membantu mahasiswa memahami penerapan algoritma dan logika pemrograman secara sederhana.

2.3 Cara Kerja Program

Program bot pada Robocode Tank Royale bekerja secara otomatis berdasarkan kondisi arena pertandingan. Bot akan selalu melakukan pengamatan dan pengambilan keputusan selama pertandingan berlangsung.

Alur kerja program bot secara umum :

1. Bot dijalankan pada arena pertandingan
Sistem akan memuat bot ke dalam arena Robocode Tank Royale
2. Radar melakukan pemindaian musuh
Bot menggunakan radar untuk mendeteksi posisi, arah, dan jarak musuh.
3. Data musuh diproses

- Informasi hasil pemindaian digunakan sebagai input untuk menentukan strategi.
4. Algoritma greedy memilih aksi terbaik
Bot menemukan tindakan yang dianggap paling menguntungkan berdasar heuristic tertentu.
 5. Bot melakukan aksi
Tindakan seperti bergerak, menghindar, memutar radar, atau menembak.
 6. Proses diulang secara terus-menerus
Bot akan terus memperbarui keputusan berdasarkan informasi yang didapat sampai pertandingan selesai.

2.4 Implementasi Algoritma Greedy Pada Bot

Penerapan algoritma greedy pada bot dilakukan dengan memilih tindakan terbaik berdasarkan kondisi yang sedang terjadi di arena pertandingan. Bot mengambil keputusan secara langsung dengan menghitung kemungkinan gerakan di masa depan. Dalam perwujudannya, bot menggunakan heuristic atau aturan tertentu untuk menentukan aksi yang paling menguntungkan. Heuristic juga digunakan sebagai dasar pengambilan keputusan secara cepat.

Beberapa contoh penerapan algoritma greedy pada bot antara lain:

1. Memilih musuh dengan jarak terpendek
Bot akan menyerang musuh dengan jarak paling dekat karena memiliki peluang terkena tembakan yang lebih tinggi.
2. Menembak saat peluang mengenai target tinggi
Bot menembak ketika arah meriam telah sesuai dengan posisi target musuh.
3. Bergerak menuju posisi aman
Bot memilih area arena yang minim ancaman serangan.
4. Menghindari peluru yang mendekat
Bot segera bergerak menghindar ketika mendeteksi serangan lawan.
5. Mengatur prioritas target
Bot dapat memilih target berdasarkan energi musuh, jarak, atau tingkat ancaman.

Strategi greedy sangat cocok digunakan pada Robocode Tank Royale karena pertandingan berlangsung cepat sehingga bot membutuhkan pengambilan keputusan secara real-time atau langsung dengan proses komputasi yang ringan.

2.5 Heuristic pada Algoritma Greedy

Heuristic merupakan aturan atau pendekatan yang digunakan untuk menentukan keputusan terbaik pada algoritma greedy. Dalam konteks permainan Robocode Tank Royale, heuristic digunakan agar bot dapat memilih keputusan tindakan secara cepat dan efisien.

Beberapa heuristic yang umum digunakan pada bot antara lain:

- Jarak musuh

- Besar energi bot
- Posisi terhadap dinding arena
- Kecepatan gerak musuh
- Arah peluru lawan
- Tingkat akurasi tembakan

Sebagai contoh:

- Jika musuh berada dekat, bot akan lebih agresif menyerang.
- Jika energi bot rendah, bot akan lebih fokus menghindar.
- Jika bot terlalu dekat dengan dinding, bot akan berpindah posisi agar tidak terjebak.

Pemilihan heuristic sangat memengaruhi performa bot selama pertandingan berlangsung.

2.6 Bahasa Pemrograman yang Digunakan

Pada implementasi bot Robocode Tank Royale, bahasa pemrograman yang digunakan adalah bahasa pemrograman C# dengan platform .NET.

C# merupakan bahasa pemrograman berorientasi objek yang dikembangkan oleh Microsoft. Bahasa ini kami pilih karena memiliki sintaks yang terstruktur, mudah dipahami, dan mendukung pengembangan aplikasi berbasis object-oriented programming.

Framework .NET digunakan untuk proses:

- Kompilasi program
- Manajemen library
- Eksekusi bot
- Integrasi dengan Robocode Tank Royale

2.7 Cara Menjalankan Bot

Langkah menjalankan bot pada Robocode Tank Royale adalah sebagai berikut:

1. Menginstall Robocode Tank Royale.
2. Menginstall bahasa pemrograman dan SDK yang digunakan.
3. Menjalankan server Robocode Tank Royale.
4. Mengcompile source code bot.
5. Menjalankan pertandingan melalui arena Robocode.

Contoh perintah menjalankan bot menggunakan .NET:

```
dotnet build
```

```
dotnet run
```

Setelah bot berhasil dijalankan, bot akan muncul pada daftar robot di arena Robocode Tank Royale dan siap digunakan dalam pertandingan.

BAB 3

APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode ke Elemen Algoritma Greedy

BotNAHUBB diimplementasikan menggunakan pendekatan Algoritma Greedy, dimana setiap giliran (turn) bot mengambil keputusan terbaik secara lokal berdasar informasi yang tersedia pada saat itu. Berikut ini merupakan pemetaan elemen greedy pada BotNAHUBB :

3.1.1 Himpunan Kandidat

Himpunan kandidat merupakan seluruh aksi yang dapat dilakukan bot pada setiap *turn*. Aksi tersebut meliputi:

- Bergerak maju atau mundur dengan $\text{TargetSpeed} = \pm 8$
- Mengubah arah badan bot menggunakan TurnRate
- Memutar arah radar untuk mendeteksi musuh
- Memutar arah gun menuju target menggunakan GunTurnRate
- Menembak dengan power tertentu
- Melakukan evasive movement atau gerakan zig-zag untuk menghindari peluru

3.1.2 Himpunan Solusi

Himpunan solusi merupakan kumpulan aksi yang dipilih bot secara bertahap selama pertandingan berlangsung. Setiap aksi dipilih berdasarkan hasil evaluasi heuristic terbaik pada kondisi saat itu.

3.1.3 Fungsi Seleksi

Fungsi seleksi digunakan untuk memilih aksi terbaik berdasarkan nilai heuristic tertentu.

Contoh:

- Memilih posisi paling aman dari serangan musuh
- Memilih arah gerakan untuk menghindari peluru
- Memilih target dengan peluang hit terbesar
- Menggunakan predictive aiming untuk memprediksi posisi musuh

3.1.4 Fungsi Kelayakan

Fungsi kelayakan digunakan untuk memastikan bahwa aksi yang dipilih dapat dilakukan oleh bot.

Contoh:

- Bot tidak menabrak dinding arena

- Energi cukup untuk menembak
- Posisi pergerakan masih aman
- Target masih berada dalam jangkauan radar dan gun.

3.1.5 Fungsi Objektif

Tujuan utama strategi greedy pada BotNAHUBB adalah:

- Memaksimalkan damage terhadap musuh
- Meminimalkan damage yang diterima
- Meningkatkan akurasi tembakan
- Bertahan hidup selama mungkin

3.1.6 Fungsi Solusi

Solusi dianggap selesai ketika ronde pertandingan berakhir atau bot telah dikalahkan.

3.2 Alternatif Strategi Greedy

3.2.1 Aggressive Greedy (Bot ClosestHunter)

Heuristic:

Memilih aksi dengan jarak terdekat untuk melakukan serangan dan ram terhadap musuh.

Karakteristik:

- Selalu mendekati musuh
- Menembak dengan power besar
- Menabrak musuh secara terus-menerus

Kelebihan:

- Menekan musuh dengan cepat

Kekurangan:

- Rentan terkena serangan balik
- Kurang efektif menghadapi banyak musuh

3.2.2 Survival Greedy (Bot Kabur)

Heuristic:

Memilih arah pergerakan yang paling jauh dari musuh dan serangan lawan.

Karakteristik:

- Banyak bergerak menghindar
- Menjaga jarak dari musuh
- Mengutamakan bertahan hidup

Kelebihan:

- Lebih sulit terkena peluru

Kekurangan:

- Damage yang dihasilkan lebih kecil
- Sulit mengalahkan musuh dengan cepat

3.2.3 Defensive Evasive Strategy (Bot Push)

Heuristic:

Memilih pergerakan maju mundur untuk menghindari serangan sambil mempertahankan jarak aman.

Karakteristik:

- Bergerak maju mundur atau jiggle movement
- Menjaga jarak aman dari musuh
- Menghindari tembakan lawan
- Menembak ketika peluang hit cukup baik

Kelebihan:

- Memiliki kemungkinan terkena serangan yang lebih rendah

Kekurangan:

- Serangan cenderung lebih lambat

3.2.4 Predictive Evasive Strategy (Bot NAHUBB)

Heuristic:

Memilih pergerakan paling aman berdasarkan arah serangan musuh serta memprediksi posisi target untuk meningkatkan akurasi tembakan.

Karakteristik:

- Menghindari peluru berdasarkan arah tembakan musuh
- Menggunakan predictive aiming
- Menyesuaikan arah gerak secara dinamis
- Menembak berdasarkan prediksi posisi target

Kelebihan:

- Akurasi tembakan lebih tinggi
- Lebih sulit terkena serangan

Kekurangan:

- Perhitungan lebih kompleks
- Kurang efektif jika prediksi meleset

3.3 Analisis Efisiensi dan Efektivitas

Strategi	Kelebihan	Kekurangan
Aggressive	Damage besar dan mampu menekan musuh dengan cepat	Mudah terkena serangan balik
Survival	Bertahan hidup lebih lama	Damage yang dihasilkan kecil
Defensive Evasive	Sulit terkena peluru karena jiggle movement	Serangan cenderung lebih lambat
Predictive Evasive	Akurasi tinggi dan sulit terkena serangan	Perhitungan lebih kompleks

3.4 Strategi yang Dipilih

Strategi yang dipilih untuk diimplementasikan adalah Predictive Evasive Strategy.

Alasan pemilihan:

- Memiliki kemampuan menghindari serangan dengan lebih baik.
- Menggunakan predictive aiming sehingga akurasi tembakan lebih tinggi.
- Mampu menjaga jarak aman sambil tetap memberikan serangan kepada lawan.
- Pergerakan lebih sulit diprediksi karena menggunakan evasive movement.
- Memiliki performa yang lebih stabil dibanding strategi lain.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi

4.1.1 Implementasi Metode *DoMovement* pada Kelas Bot NAHUBB

Procedure DoMovement ()
{ Menentukan gerakan bot setiap turn berdasarkan posisi dan kondisi }

Kamus Lokal

centerAngle : real { sudut menuju tengah arena }
turnNeeded : real { besar belokan yang diperlukan }

Algoritma

```
if IsNearWall() then
    centerAngle ← AngleTo(ArenaWidth / 2, ArenaHeight / 2)
    turnNeeded ← Normalize(centerAngle - Direction)
    TurnRate ← Sign(turnNeeded) × Min(|turnNeeded|, 10)
    TargetSpeed ← 8 × moveDirection
    Return

{ Gerak sinusoidal agar sulit ditembak }
TargetSpeed ← Sin(TurnNumber × 0.13) × 8 × moveDirection

if movementCounter mod (45 + rand(0..19)) = 0 then
    moveDirection ← moveDirection × -1 { balik arah secara berkala }

if movementCounter mod (30 + rand(0..14)) = 0 then
    TurnRate ← rand(-8..8) { ganti sudut belokan acak }
```

4.1.2 Implementasi Metode *DoGunAndRadar* pada kelas Bot NAHUBB

Procedure DoGunAndRadar ()

{ Menjalankan logika radar lock, predictive aiming, dan penembakan }

Kamus Lokal

```
targetKv : KeyValuePair<int, EnemyData>
enemy : EnemyData
scanAge : integer { selisih turn sejak terakhir terlihat }
dirToEnemy : real { sudut dari bot ke musuh }
radarTurn : real
firePower : real
bulletSpeed : real
travelTime : real
predictedX : real
predictedY : real

aimAngle : real
gunTurn : real
```

Algoritma

```
{ CASE 1: Tidak ada musuh sama sekali }
if enemies.Count = 0 then
    SweepRadar()
    Return

{ Pilih target terbaik berdasarkan skor }
targetKv ← enemies.OrderBy(Distance × 0.6 + Energy ×
2.0).First()
enemy ← targetKv.Value
scanAge ← TurnNumber - enemy.LastSeen

{ CASE 3: Data musuh sudah basi → sweep aktif }
if scanAge > MaxScanAge (3) then
    SweepRadar()
    Return

{ CASE 2: Data segar → lock radar ke target }
dirToEnemy ← AngleTo(enemy.X, enemy.Y)
radarTurn ← Normalize(dirToEnemy - RadarDirection)
SetTurnRadarLeft(radarTurn × 2.0) { overshoot agar tidak
lepas }

{ Predictive aiming: perkirakan posisi musuh }
firePower ← CalculateFirePower(enemy)
bulletSpeed ← 20.0 - 3.0 × firePower
travelTime ← enemy.Distance / bulletSpeed
predictedX ← Clamp(enemy.X + Cos(enemy.Heading) × enemy.Speed
× travelTime, 20, ArenaWidth-25)
predictedY ← Clamp(enemy.Y + Sin(enemy.Heading) × enemy.Speed
× travelTime, 20, ArenaHeight-25)

aimAngle ← AngleTo(predictedX, predictedY)
gunTurn ← Normalize(aimAngle - GunDirection)
SetTurnGunLeft(gunTurn)

{ Tembak jika gun sudah lurus dan cooldown habis }
```

```

if |gunTurn| < 3° and TurnNumber - enemy.LastFiredAt ≥ 5 then
  if Energy < 30 then
    SetFire(1)
  else
    Fire(firePower)
    enemy.LastFiredAt ← TurnNumber

{ Ram jika musuh hampir mati dan sangat dekat }
if enemy.Distance < 100 and enemy.Energy < 20 then
  TargetSpeed ← 8

```

4.1.3 Implementasi fungsi *CalculateFirePower* pada kelas Bot NAHUBB

```

Function CalculateFirePower (enemy : EnemyData) → real
{ Mengembalikan nilai kekuatan tembak 1.0 - 3.0 sesuai kondisi musuh }

Algoritma
  if enemy.Energy < 15 then
    return 3.0 { musuh hampir mati → habisi }

  if enemy.Distance < 120 then
    return 3.0 { jarak sangat dekat → tembak keras }

  if enemy.Distance < 300 then
    return 2.0 { jarak menengah }

  return 1.0 { jarak jauh → hemat energi }

```

4.1.4 Implementasi fungsi *IsNearWall* pada kelas Bot NAHUBB

```

Function IsNearWall () → boolean
{ Mengembalikan true jika bot berada dalam jarak 80px dari dinding }

Algoritma
  return X < 80 or X > ArenaWidth - 80
    or Y < 80 or Y > ArenaHeight - 80

```

4.2 Struktur Data

Sebagian besar struktur data /variable pada program ini menggunakan konsep algoritma greedy.

Tabel 1. Struktur data

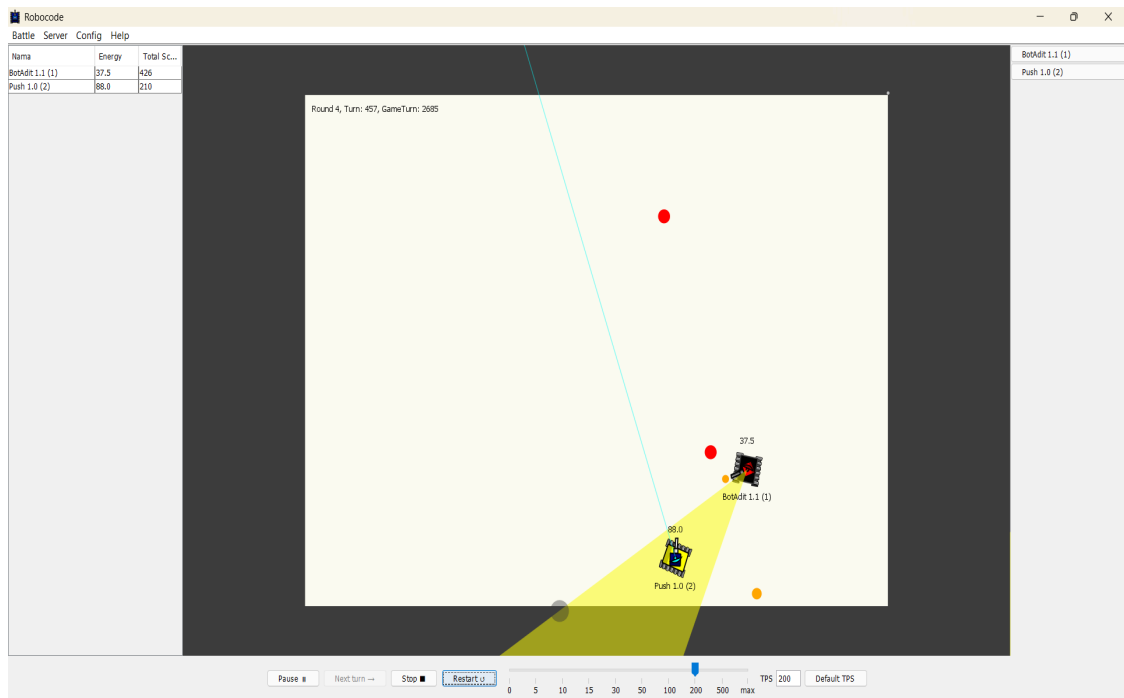
Struktur Data / Variable	Fungsi	Hubungan dengan Strategi Greedy
Enemies	Dictionary<int, EnemyData> yang menyimpan data semua musuh yang terdeteksi radar. Key = ID bot musuh, Value = objekEnemyData. Data dibersihkan otomatis jika musuh tidak terlihat lebih dari 10 turn.	Digunakan sebagai sumber data utama untuk memilih target terbaik berdasarkan skor $Distance * 0.6 + Energy * 2.0$, mendukung strategi greedy dalam penargetan.
EnemyData	Kelas nested untuk menyimpan <i>snapshot</i> kondisi musuh: posisi (X, Y) <i>Energy</i> , <i>Distance</i> , <i>Heading</i> , <i>Speed</i> , <i>LastSeen</i> , dan <i>LastFireAt</i>	Menjadi dasar seluruh keputusan greedy: pemilihan target, prediksi, posisi tembak, penentuan daya tembak, dan kondisi ram musuh lemah.
moveDirection	Menyimpan arah gerak bot: +1 (maju) atau -1 (mundur). Dibalik saat menabrak dinding, bot lain, atau terkena serangan	Digunakan untuk mengubah arah secara reaktif ketika bot menghindari dinding atau menabrak bot lain, mendukung strategi bertahan hidup.
movementCounter	Perhitungan jumlah turn yang telah berlalu. Digunakan untuk menjadwalkan pembalikan arah dan perubahan sudut belok secara periodik di DoMovement()	Memastikan gerakan bot ini bervariasi dan tidak terpola agar sulit untuk diprediksi oleh bot lain yang mendukung strategi bertahan hidup (<i>survival</i>).
rand	Pencetak angka acak (<i>Random</i>). Digunakan untuk menentukan interval balik arah, besar sudut belok, dan variasi kecepatan gerak bot NAHUBB	Menghasilkan variasi gerakan agar bot tidak mudah diprediksi dan diserang oleh bot lain untuk mendukung strategi bertahan.
radarSweepDir	Arah rotasi radar saat mode <i>sweep</i> aktif. Bernilai +1(kanan) atau -1 (kiri). Bergantian setiap 360° agar radar mengecek seluruh arena secara merata.	Memastikan radar tidak terjebak di satu sisi sehingga bot selalu mendapatkan data musuh terbaru untuk dilakukannya pengambilan keputusan secara greedy

sweepAccumulator	Akumulasi derajat rotasi radar selama mode <i>sweep</i> . Bertambah 20° setiap turn. Saat mencapai 360° , arah balik dan nilai akan direset ke 0° .	Mendukung efisiensi deteksi musuh dengan memutuskan radar berpindah secara terstruktur sehingga data musuh yang digunakan untuk pemilihan dengan greedy selalu yang baru.
MaxScanAge	Konstanta (nilai: 3) adalah batas usia data scan dalam turn. Jika TurnNumberLastSeen > 3, maka data dianggap tidak yang baru dan beralih ke mode sweep	Mencegah pengambilan data keputusan greedy berdasarkan data musuh yang sudah tidak akurat sehingga bot tidak melakukan penyerangan pada posisi yang tidak akurat tersebut.
FireAngleThreshold	Konstanta (nilai : 3.0°) adalah toleransi sudut <i>aiming</i> . Bot hanya menembak jika selisih arah senjata terhadap target lebih kecil dari nilai ini	Memastikan tembakan greedy hanya dilepaskan ketika peluang mengenai target cukup tinggi, menghemat energi dari tembakan yang meleset
FireCooldown	Konstanta (nilai : 5 turn) adalah Jeda minimum antar dua tembakan ke musuh yang sama. Diimplementasikan oleh LastFiredAt.	Mencegah pemborosan energi akibat tembakan beruntun yang tidak terkontrol yang bertujuan untuk menghemat energi bot tetap cukup untuk bertahan dan melakukan penyerangan secara optimal.
WallMargin	Konstanta (nilai : 80.0 piksel) adalah jarak minimum dari tepi arena. Jika posisi bot lebih dekat dari nilai ini ke dinding mana pun, bot akan berbelok ke tengah arena.	Menentukan batas aman pergerakan bot agar tidak terjebak di sudut arena sehingga bot dapat menghindari dari dinding untuk mendukung strategi bertahan.

4.3 Analisis Pengujian

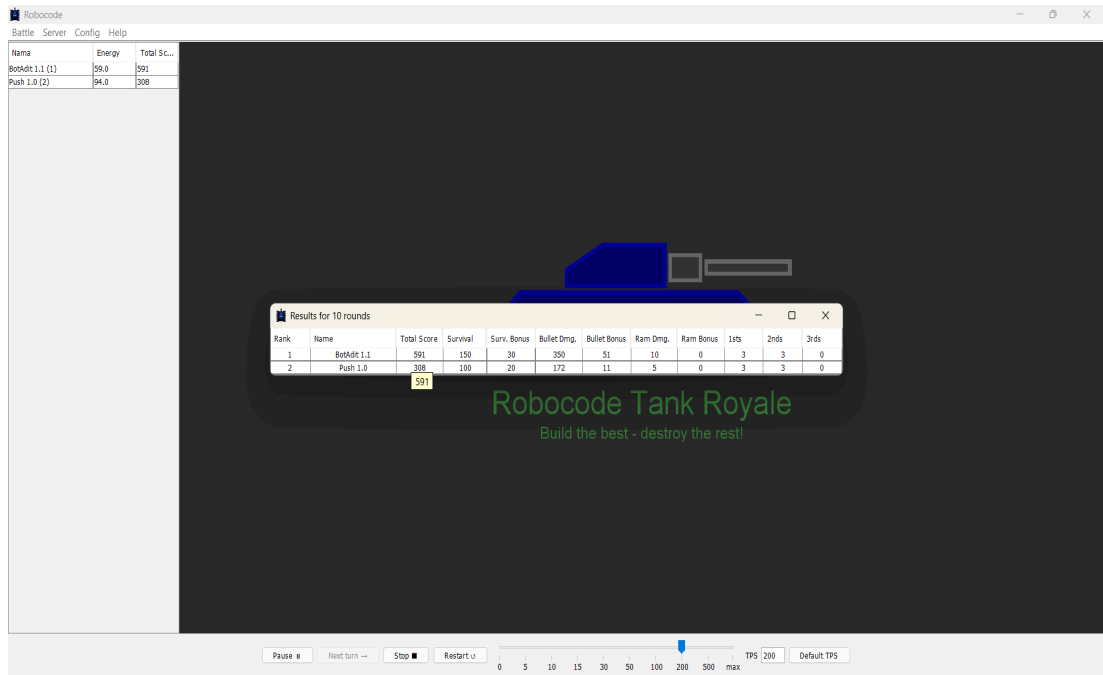
Pengujian dilakukan sebanyak 3 kali yakni mempertandingkan bot utama kelompok kami dengan 3 bot alternatif yang sudah dibuat.

4.3.1 Pengujian Ke-1



Gambar 2. Kemenangan bot NAHUBB melawan bot Push(bot Alternatif 1).

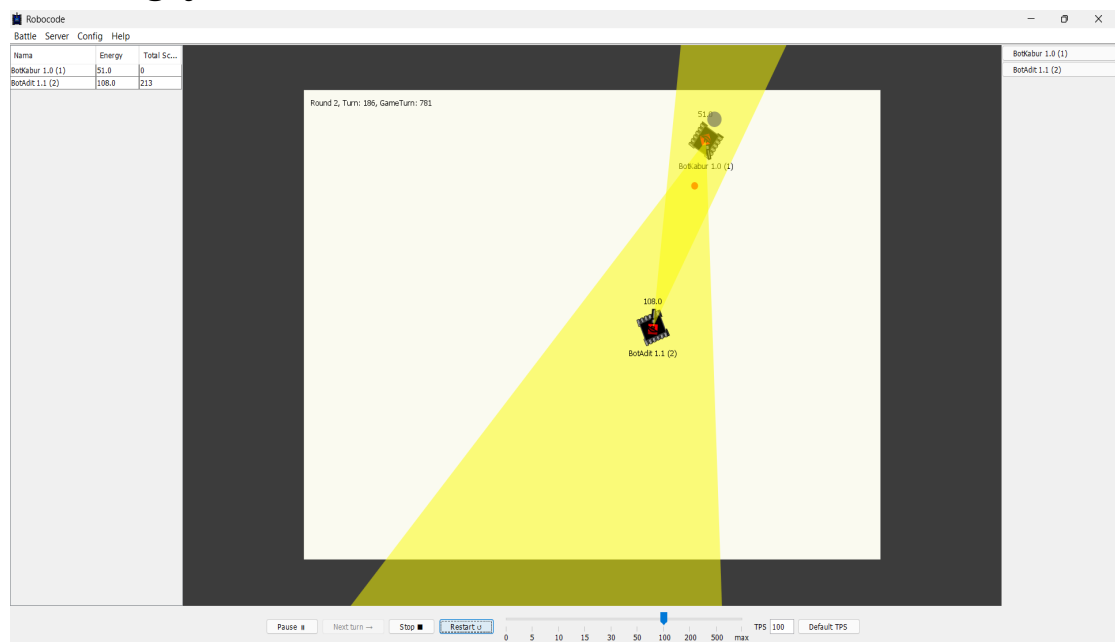
Pada pengujian ke-1, pengujian ini dilakukan sebanyak 10 round dan pada round ke 4 botNAHUBB mengungguli bermain melawan bot Push. CalculateFirePower yang digunakan bot kami membuat kemenangan melawan bot Push karena fungsi tersebut menerapkan perkondisian dengan berpacu pada *enegry* musuh dan jarak musuh untuk melakukan penyerangan. Pada gambar terserbut juga dapat dilihat bot NAHUBB sedang bermanuver untuk menghindari serangan dari bot Push. Salah satu hal yang membuat bot kami menang ialah karena pergerakan bot push yang hanya maju mundur sehingga sangat mudah untuk diserang oleh bot kami . Hal ini disebabkan karena bot kami memperkirakan posisi musuh sebelum dilakukan penyerangan.



Gambar 3. Hasil bot NAHUBB melawan bot Push(bot alternatif 1).

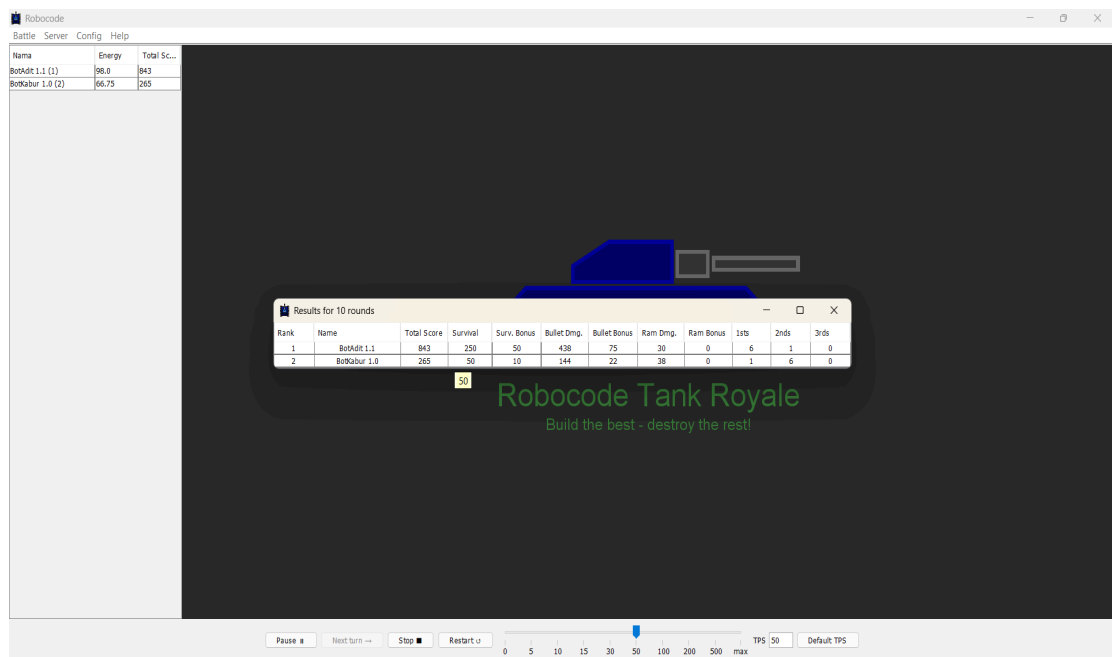
Dari hasil 10 *rounds* yang sudah dilakukan diperoleh hasil bahwa bot adit (bot NAHUBB) pemenangnya melawan bot Push dengan perbandingan skor yang cukup jauh. Pada bagian *Survival*, bot NAHUBB menang dengan bot velocity dan pada bagian *bullet damage* bot kami lebih unggul bot Push dikarenakan dapat memprediksi pergerakan musuh dan menembak sesuai dengan jarak dan *energy* musuh.

4.3.2 Pengujian Ke-2



Gambar 4. Kemenangan bot NAHUBB melawan bot Kabur (bot alternatif 2).

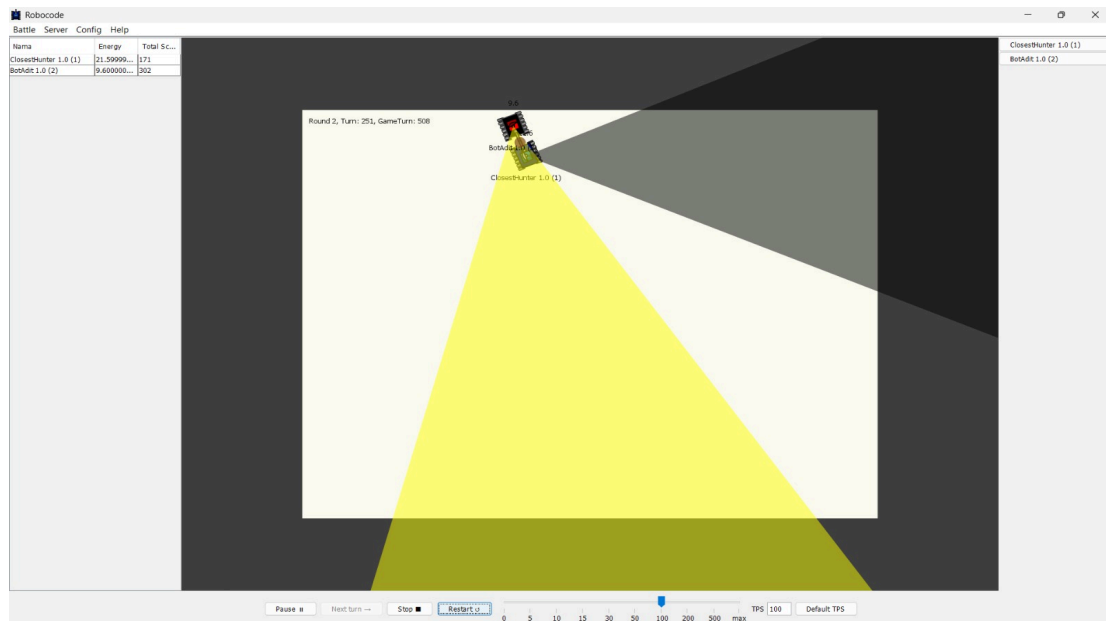
Pada pengujian ke-2, dilakukan pengujian 1 vs 1 antara bot NAHUBB dengan bot kabur. Pada pengujian ini berlangsung sangat sengit dikarenakan bot kabur adalah pengembangan dari bot NAHUBB yang di mana bot kabur tersebut terdapat algoritma untuk melarikan diri dari bot lain. Pada gambar tersebut bot NAHUBB mengungguli pertandingan pada round 2 dikarenakan bot NAHUB ini akan selalu mendekati musuhnya dan melakukan penyerangan. Hal tersebut menyebabkan bot kabur fokus untuk menjaga jarak dan tidak melakukan penyerangan.



Gambar 5. Hasil bot NAHUBB melawan bot Kabur (bot alternatif 2).

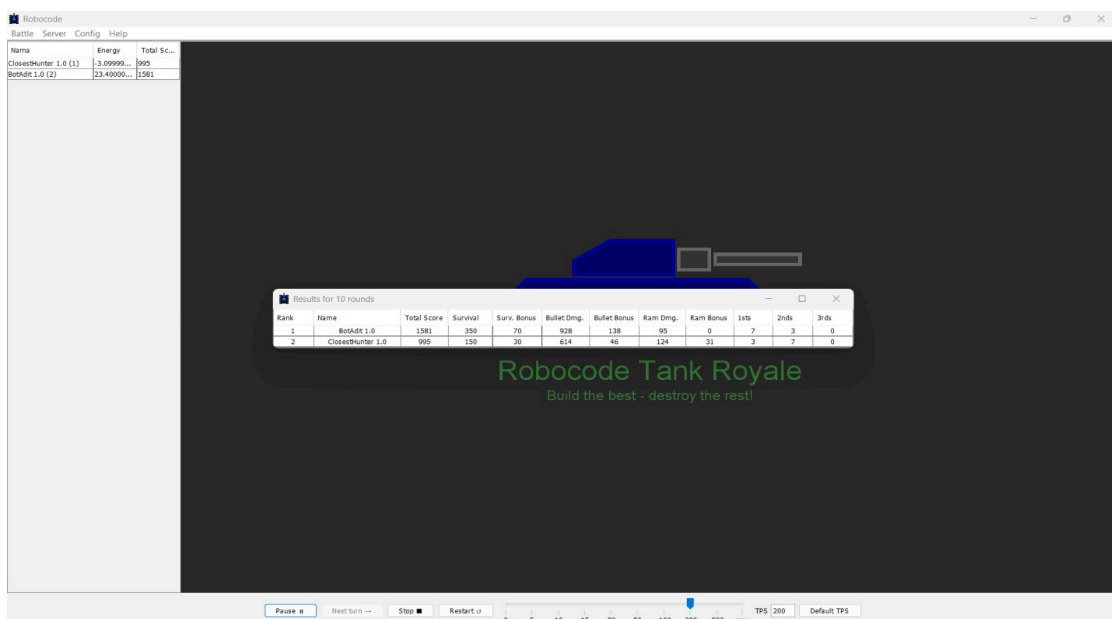
Hasil 10 *rounds* yang sudah dilakukan diperoleh hasil bahwa bot adit (bot NAHUBB) pemenangnya melawan bot Kabur dengan perbandingan skor yang cukup jauh. Pada bagian Survival, bot NAHUBB tetap menang yang seharusnya bot Kabur mengungguli hal ini dikarenakan bot NAHUBB terlalu *agresif* dalam melancarkan serangan dibantu dengan adanya fungsi prediksi penyerangan, sehingga pada bagian *bullet damage* bot kami lebih mengungguli bot Push dikarenakan dapat memprediksi pergerakan musuh dan menembak sesuai dengan jarak dan *energy* musuh.

4.3.3 Pengujian Ke-3



Gambar 6. Kemenangan bot NAHUBB melawan bot ClosetsHunter (bot alternatif 3)

Pada pengujian ke-3, dilakukan lagi pertandingan melawan bot ClosetsHunter yang berbeda dari sebelumnya. Pada pengujian kali ini bot NAHUBB juga mengungguli terhadap bot ClosestHunter. Bot ClosestHunter ini adalah bot pengembangan dari bot Ram Fire tetapi lebih agresif karena bot tersebut bukan hanya *ram* saja, tetapi juga melakukan *fire* terhadap musuh. Namun, hal tersebut dalam di-*handle* dengan baik oleh bot NAHUBB dikarenakan ada fungsi *CalculateFirePower*. Jika jarak musuh kurang dari 15 dari bot NAHUBB, maka akan melancarkan fire sebesar 3 sehingga dapat dengan mudah di-*handle*.



Gambar 7. Hasil bot NAHUBB melawan bot ClosetsHunter (bot alternatif 3)

10 rounds yang sudah dilakukan diperoleh hasil dengan skor yang diperoleh bot adit (bot NAHUBB) 1581 dan bot ClosetsHunter 995, sehingga bot NAHUBB dinyatakan menang. Apabila ditinjau dari *survival* dan *Bullet damage* bot NAHUBB mengungguli, tetapi dalam *Ram damage* bot ClosestHunter mengungguli yang sesuai dengan maksimasinya.

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian, Bot NAHUBB dapat mengungguli dari 3 bot alternatif yang sudah dibuat dengan menerapkan *Predictive Evasive Strategy* greedy yang di mana dapat menghindari serangan dengan baik, menggunakan prediksi aiming sehingga akurasi tembakan lebih tinggi, dan mampu menjaga jarak aman sambil tetap memberikan serangan kepada lawan sehingga lebih sulit diprediksi. Namun kelemahan dari konsep tersebut jika pergerakan dari bot musuh tersebut memiliki pergerakan acak maka akan sulit untuk memprediksi pergerakannya dan bot tersebut menggunakan strategi bertahan.

Pada pengujian pertama dengan bot push, bot NAHUBB mengungguli karena pergerakan dari bot push tersebut terbaca karena hanya maju mundur sehingga sangat mudah terkena serangan dari bot NAHUBB. Pada Pengujian kedua dengan bot Kabur, bot NAHUBB memenangkan pertarungan karena bot NAHUBB selalu mendekati bot Kabur dan melancarkan serangan sedangkan bot Kabur fokus untuk manjauh dari bot NAHUBB. Pada pengujian ketiga dimenangkan kembali oleh Bot NAHUBB melawan bot alternatif 3 yaitu bot ClosestHunter. Bot tersebut dapat dengan mudah di-*handle* oleh bot NAHUBB dengan CalculateFirePower dikarenakan bot ClosetsHunter tersebut untuk melakukan *Ram*. Oleh karena itu, berdasarkan pengujian yang sudah dilakukan kami milih bot NAHUBB menjadi bot utama dengan menerapkan konsep *Greedy Predictive Evasive Strategy* yang berfokus pada maksimasi pada prediksi penyerangan dan minimasi terkena serangan (*survival*) dalam sebuah pertandingan.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, algoritma *greedy* berhasil diterapkan dengan baik dalam sistem pengambilan keputusan taktis pada bot *Robocode Tank Royale*. Penelitian ini berhasil merancang dan menganalisis empat alternatif strategi berbasis *greedy*, di mana strategi *aggressive greedy* ditemukan memiliki performa terbaik dalam situasi pertandingan 1 vs 1 karena mampu memberikan tekanan secara konstan dan terus-menerus kepada pihak lawan. Melalui seluruh rangkaian uji coba tersebut, bot NAHUBB akhirnya dipilih sebagai bot utama dengan mengintegrasikan konsep *Greedy Predictive Evasive Strategy*. Strategi pada bot utama ini berfokus pada maksimasi akurasi fungsi prediksi penyerangan sekaligus minimasi risiko terkena tembakan demi menjaga ketahanan hidup (*survival*) bot di dalam arena pertandingan. Terakhir, efektivitas dan ketepatan pengambilan keputusan taktis pada bot ini terbukti sangat dipengaruhi oleh kualitas dan formulasi fungsi *heuristic* yang digunakan.

5.2 Saran

Untuk pengembangan atau penelitian bot simulasi pertempuran ini lebih lanjut di masa mendatang, disarankan beberapa hal sebagai berikut:

1. **Hibridisasi Algoritma:** Menggabungkan pendekatan algoritma *greedy* dengan algoritma pencarian atau kecerdasan buatan lainnya yang lebih kompleks, seperti algoritma *minimax* atau pemanfaatan *machine learning* (pembelajaran mesin).
2. **Peningkatan Prediksi Gerak:** Menambahkan kalkulasi tingkat lanjut untuk memprediksi arah pergerakan dan posisi masa depan dari robot musuh secara lebih akurat.
3. **Pengembangan Strategi Adaptif:** Merancang strategi bot yang dinamis dan adaptif sehingga perilakunya dapat berubah secara otomatis menyesuaikan situasi riil serta kondisi tak terduga di dalam arena pertandingan.
4. **Optimasi Fitur Evasive:** Mengoptimalkan pola gerakan menghindar (*evasive movement*) milik tank agar bot menjadi jauh lebih lincah dan sulit untuk dibidik atau terkena peluru yang ditembakkan lawan.

LAMPIRAN

Link Repositori : <https://github.com/Kepinn-12/Tubes-NAHUBB.git>

Link Youtube : <https://youtu.be/bF0JLL8PLvE>

Tabel 2. Ketersediaan

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada Youtube.	✓	

DAFTAR PUSTAKA

- [1] R. Munir, "Algoritma Greedy," slide perkuliahan, Program Studi Teknik Informatika, Institut Teknologi Bandung, Bandung, Indonesia, 2025.
- [2] S. Oktaviana and A. Naufal, "Algoritma Greedy untuk Optimalisasi Ruang dalam Penyusunan Jadwal Perkuliahan," *Jurnal Multinetics*, vol. 3, no. 1, pp. 52-56, Mei 2017.
- [3] F. N. Larsen, "Getting Started," *Robocode Tank Royale Docs*, 2022. [Online]. Available: <https://robocode.dev/tutorial/getting-started.html>. [Accessed: May 21, 2026].